

Functional Go

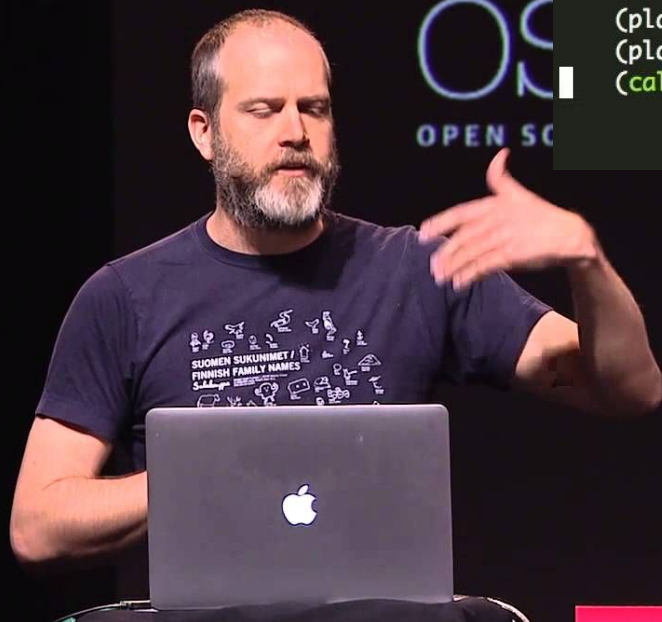
Why would you do that?



Quicksort in Haskell

```
quicksort [] = []  
quicksort (x:xs) =  
    let smaller = quicksort [a | a <- xs, a <= x]  
        bigger  = quicksort [a | a <- xs, a > x]  
    in  smaller ++ [x] ++ bigger
```

Live coding



```
(define left-hand
  (lambda (beat ps ds)
    (if (= 0 (modulo beat 8))
        (set! root (random (remove root (list 48 43)))))
    (play sampler (car ps) 80 12.0) ;;(car ds))
    (play 1/2 sampler root 80 12.0)) ;;(car ds))
    (callback (*metro* (+ beat (* .95 (car ds)))) 'left-hand (+ beat (car ds))
              (rotate ps -1)
              (rotate ds -1))))
```

Andrew Sorensen Keynote at OSCON

**What is functional
programming?**

Functional programming

- no mutable state
- functions as first class objects

No mutable state

- simply a choice
- no for loops

Iterative sum

```
func SumI(vs []int) int {  
    s := 0  
    for _, v := range vs {  
        s += v  
    }  
    return s  
}
```

Iterative fun is not functional

```
func SumI(vs []int) int {  
    s := 0  
    for _, v := range vs {  
        s += v  
    }  
    return s  
}
```

Recursive sum

```
func SumR(vs []int) int {  
    if len(vs) == 0 {  
        return 0  
    }  
    return vs[0] + Sum(vs[1:])  
}
```

Is it fast?

PASS

BenchmarkSumI-4	3000000	462 ns/op
BenchmarkSumR-4	300000	4707 ns/op

Tail recursion

```
func SumTR(vs []int, s int) int {  
    if len(vs) == 0 {  
        return s  
    }  
    return Sum(vs[1:], s + vs[0])  
}
```

Is it fast?

PASS

BenchmarkSumI-4	3000000	462 ns/op
BenchmarkSumR-4	300000	4707 ns/op
BenchmarkSumTR-4	300000	5056 ns/op

Tail recursion with faked optimization

```
func SumTRG(vs []int, s int) int {  
begin:  
    if len(vs) == 0 {  
        return s  
    }  
    vs, s = vs[1:], s + vs[0]  
    goto begin  
}
```

Is it fast?

PASS

BenchmarkSumI-4	3000000	462 ns/op
BenchmarkSumR-4	300000	4707 ns/op
BenchmarkSumTR-4	300000	5056 ns/op
BenchmarkSumTRG-4	1000000	1587 ns/op

This is not about the speed, clearly

Functions are first class objects

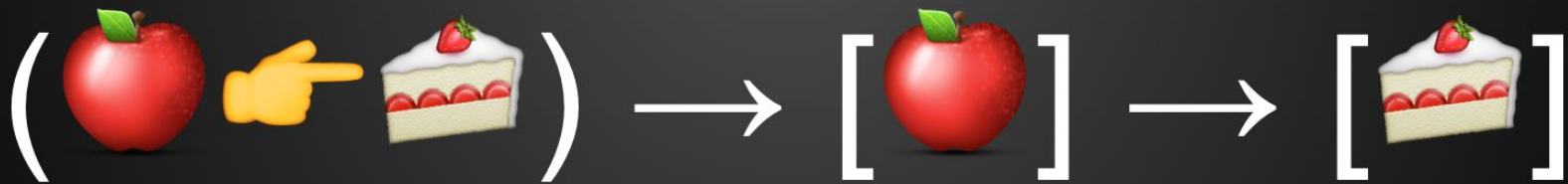
- as parameters
- as returned values
- in Go they can be used anywhere!

High order functions

Functions applied on functions

- map, filter, fold, etc

map





$\text{🍏} = \text{int}$

$\text{🍰} = \text{bool}$

Map on concrete types

```
func Map(f func(v int) bool, vs []int) []bool {  
    if len(vs) == 0 {  
        return nil  
    }  
  
    return append(  
        []bool{f(vs[0])},  
        Map(f, vs[1:])...)  
}
```

Map on concrete types (reversed order)

```
func Map(f func(v int) bool, vs []int) []bool {  
    if len(vs) == 0 {  
        return nil  
    }  
  
    return append(  
        Map(f, vs[:len(vs)-1]),  
        vs[len(vs)-1])  
}
```

Map on concrete types

```
func main() {  
    isEven := func(v int) bool { return v%2 == 0 }  
    nums := []int{1, 2, 3, 4, 5, 6}  
    fmt.Println(Map(isEven, nums))  
    // [false true false true false true]  
}
```

If Go had generics

template<, >

func Map(f func(a ) , vs []) [])

But it doesn't

func(a )  ⇒ interface{}

[]  ⇒ interface{}

[]  ⇒ interface{}

```
func Map(f interface{}, vs interface{}) interface{}
```

```
func Map(f interface{}, vs interface{}) interface{}
```



“embrace the interface”

but not too much

```
func Map(f interface{}, vs interface{}) interface{}
```

Representing functions

```
type Func struct {  
    in reflect.Type  
    out reflect.Type  
    f func(interface{}) interface{}  
}
```

```
func (f Func) Call(v interface{}) interface{} {  
    return f.f(v)  
}
```

Representing functions: NewFunc

```
func NewFunc(f interface{}) (*Func, error) {  
    // check type of f and return an error if needed  
    return &Func{  
        in: tf.In(0), out: tf.Out(0),  
        f: func(x interface{}) interface{} {  
            out := vf.Call([]reflect.Value{reflect.ValueOf(x)})  
            return out[0].Interface()  
        },  
    }, nil  
}
```

Representing functions: Must

```
func Must(f *Func, err error) *Func {  
    if err != nil {  
        panic(err)  
    }  
    return f  
}
```

```
func Map(f interface{}, vs interface{}) interface{}
```



```
func Map(f *Func, vs interface{}) interface{}
```

func Map(f *Func, vs interface{}) interface{}

A simple List

```
type List struct {  
    Head interface{}  
    Tail *List  
}
```

```
func Map(f *Func, l *List) *List {  
    if l == nil { return nil }  
    return &List{f.Call(l.Head), Map(f, l.Tail)}  
}
```

Should this be a method? Of what?

Map in Go, as a method of List

```
func (l *List) Map(f *Func) *List {  
    if l == nil {  
        return nil  
    }  
    return &List{  
        f.Call(l.Head),  
        Map(f, l.Tail),  
    }  
}
```

Using map

```
toUpper := Must(NewFunc(strings.ToUpper))

m := &List{"hello", &List{"bye", nil}}
res := m.Map(toUpper)
fmt.Println(res)
// "HELLO", "BYE"
```

typeclasses: an interlude

The Show typeclass

```
class Show a where  
  show :: a -> String
```

The Stringer interface

```
type Stringer interface {  
    String() string  
}
```

The Eq typeclass

```
class Eq a where  
    (==) :: a -> a -> Bool  
    (/=) :: a -> a -> Bool
```

The Equatable interface

```
type Equatable interface {  
    Equals(e Equatable) bool  
}
```

satisfied by an Integer type

```
func (i Integer) Equals(j Integer) bool
```

typeclasses are “like” interfaces

What else can we map over?

What can we map over?

Lists

Maps

Trees

...

Mapper interface

```
func (l *List) Map(f *Func) *List
```

```
type Mapper interface {  
    Map(*Func) ???  
}
```

Functor typeclass

```
class Functor f a where  
    fmap :: (a -> b) -> f a -> f b
```

fmap



A couple of useful functors

maybe

Maybe

```
type Maybe struct {  
    Value interface{  
}  
  
func (m Maybe) Map(f *Func) Maybe {  
    if m.Value == nil {  
        return Maybe{}  
    }  
    return Maybe{ f.Call(m.Value) }  
}
```

Maybe

Once we create a Func:

```
toUpper := Must(NewFunc(strings.ToUpper))
```

We can call it with a value:

```
m := Maybe{"hello"}  
res := m.Map(toUpper)  
fmt.Println(res.Value)  
// "HELLO"
```

Maybe

Once we create a Func:

```
toUpper := Must(NewFunc(strings.ToUpper))
```

Or without it:

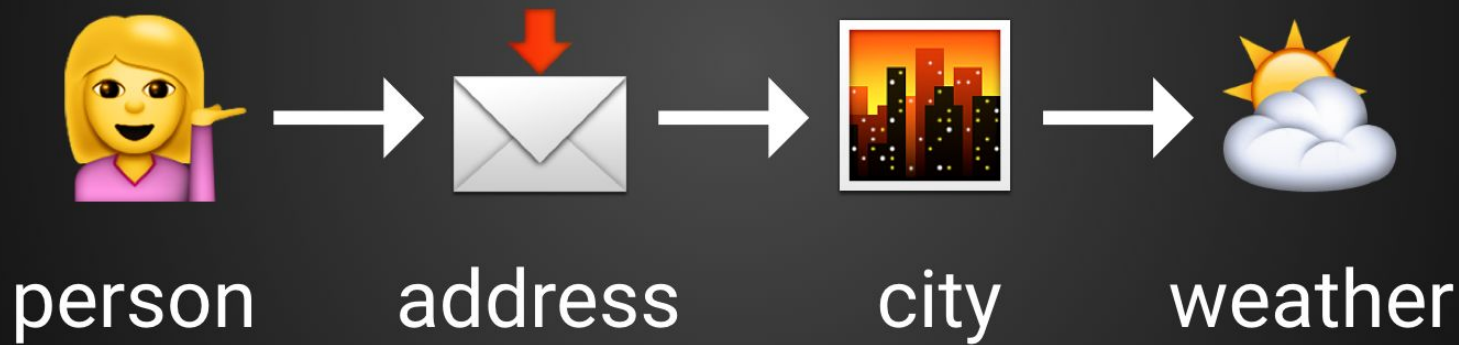
```
m := Maybe{}  
res := m.Map(toUpper)  
fmt.Println(res.Value)  
// <nil>
```

Maybe chained

```
toUpper := Must(NewFunc(strings.ToUpper))
twice := Must(NewFunc(func(s string) string { return s+s }))

m := Maybe{"hello"}
res := m.Map(toUpper).Map(twice)
fmt.Println(res.Value)
// "HELLOHELLO"
```

An actual use case



An actual use case

```
type Person struct{ address *Address }  
type Address struct{ city *City }  
type City struct{ weather *Weather }  
type Weather struct{ desc string }
```

```
func (p Person) Address() *Address { return p.address }  
func (a Address) City() *City { return a.city }  
func (c City) Weather() *Weather { return c.weather }  
func (w Weather) Desc() string { return w.desc }
```

An actual use case

```
func (p Person) Weather() string {  
    a := p.Address()  
    if a == nil { return "no weather" }  
    c := a.City()  
    if c == nil { return "no weather" }  
    w := c.Weather()  
    if w == nil { return "no weather" }  
    return w.Description()  
}
```

Maybe chained

```
func (p Person) Weather() string {  
    m := Maybe{p}.  
        Map(Must(NewFunc(func(p Person) *Address {  
            return p.address  
        }))).  
        Map(Must(NewFunc(func(a Address) *City {  
            return a.city  
        }))).  
    ...  
}
```

Method values

We can obtain a function from a value:

```
var p Person  
p.Address      // func() *Address
```

Or from a type

```
Person.Address // func(Person) *Address
```

Maybe chained

```
func (p Person) Weather() string {  
    w := Maybe{p}.  
        Map(Must(NewFunc(Person.Address))).  
        Map(Must(NewFunc(Address.City))).  
        Map(Must(NewFunc(City.Weather))).  
        Map(Must(NewFunc(Weather.Description)))  
    if w.Value == nil { return "no weather" }  
    return w.Value.(string)  
}
```

Chaining map with Maybe.Do

```
func (m Maybe) Do(fs ...interface{}) (Maybe, error) {  
    if len(fs) == 0 {  
        return m, nil  
    }  
    f, err := NewFunc(fs[0])  
    if err != nil {  
        return Maybe{}, err  
    }  
    return m.Map(f).Do(fs[1:]...)  
}
```

Chaining map with Maybe.Do

```
func (p Person) Weather() string {  
    w := Maybe{p}.Do(  
        Person.Address,  
        Address.City,  
        City.Weather,  
        Weather.Description)  
  
    if w.Value == nil { return "no weather" }  
    return w.Value.(string)  
}
```

many

Many

```
type Many struct {  
    Head interface{}  
    Tail *Many  
}
```

Many

```
func (m *Many) Map(f *Func) *Many {  
    if m == nil { return nil }  
    res := m.Tail.Map(f)  
  
    // append all elements from the result  
    for _, v := range toSlice(f.Call(m.Head)) {  
        r = &Many{v, res}  
    }  
    return res  
}
```

Many

Once we create a Func:

```
toUpper := Must(NewFunc(strings.ToUpper))
```

We can call it with a value:

```
m := Many{"hello there", "good bye"}
```

```
res := m.Map(toUpper)
```

```
fmt.Println(res.Value)
```

```
// "HELLO THERE", "GOOD BYE"
```

Many chained

If we have a new function:

```
toUpper := Must(NewFunc(strings.ToUpper))  
fields := Must(NewFunc(strings.Fields))
```

Or without it:

```
m := Many{"hello there", "good bye"}  
res := m.Map(toUpper).Map(fields)  
fmt.Println(res.Value)  
// "HELLO", "THERE", "GOOD", "BYE"
```

An actual use case



library



books



pages



lines

An actual use case

```
type Library struct{ books []Book }
```

```
type Book      struct{ pages []Page }
```

```
type Page      struct{ lines []Line }
```

```
type Line      struct{ text  string }
```

```
func (l Library) Books() []Book { return l.books }
```

```
func (b Book) Pages() []Page    { return b.pages }
```

```
func (p Page) Lines() []Line    { return p.lines }
```

```
func (l Line) Text() string      { return l.text }
```

An actual use case

```
words := make(map[string]int)
for _, b := range l.Books() {
    for _, p := range b.Pages() {
        for _, l := range p.Lines() {
            for _, word := range strings.Fields(l.Text()) {
                words[word]++
            }
        }
    }
}
```

Many chained

```
NewMany(1).  
    Map(Must(NewFunc(Library.Books))).  
    Map(Must(NewFunc(Book.Pages))).  
    Map(Must(NewFunc(Page.Lines))).  
    Map(Must(NewFunc(Line.Text))).  
    Map(Must(NewFunc(strings.Fields))).  
    Map(Must(NewFunc(func(s string) bool {  
        count[s]++;  
        return true  
    }))))
```

Many Do

```
_ , err := NewMany(m).Do(  
    Library.Books,  
    Book.Pages,  
    Page.Lines,  
    Line.Text,  
    strings.Field,  
    func(s string) bool { count[s]++; return true })  
if err != nil { // type error }
```

Many Do

```
w, err := NewMany(m).  
    Do(  
        Library.Books,  
        Book.Pages,  
        Page.Lines,  
        Line.Text,  
        strings.Field)  
if err != nil { // type error }  
w.Each(func(s string) { count[s]++ })
```

Functional Go

- Doable
 - but slower than normal code
 - requires reflection
- Inspiration for good APIs
 - functors to abstract function application
 - what could we do with monads?

Merci

@francesc

Composition of functions

```
func Compose(f, g *Func) (*Func, error) {  
    if g.out != f.in {  
        return nil, fmt.Errorf("can't compose: %v != %v", g.out, f.in)  
    }  
    return &Func{  
        g.in,  
        f.out,  
        func(x interface{}) interface{} { return f.Call(g.Call(x)) },  
    }, nil  
}
```

Mapping over a Maybe and handling nil pointers

```
func (m Maybe) Map(f *Func) Maybe {  
    if m.Value == nil { return Maybe{} }  
    r := f.Call(m.Value)  
    vr := reflect.ValueOf(r)  
    if vr.Kind() == reflect.Ptr && vr.IsNil() {  
        return Maybe{}  
    }  
    return Maybe{r}  
}
```